TourGuide

TOURGUIDE

Documentation fonctionnelle et technique

Sommaire

1. Présentation du projet
   - 1.1 Objectifs du projet
   - 1.2 Hors du champ d'application
   - 1.3 Mesures du projet
2. Spécifications fonctionnelles
3. Spécifications techniques
   - 3.1 Schémas de conception technique
   - 3.2 Glossaire
   - 3.3 Solutions techniques
   - 3.4 Autres solutions non retenues

1. Présentation du projet

TourGuide est une application SpringBoot qui permet aux utilisateurs de visualiser les attractions touristiques à proximité, et d'obtenir des réductions sur les séjours à l'hôtel ainsi que sur les billets de spectacles.

Le public cible inclut à la fois les utilisateurs finaux (voyageurs qui cherchent des activités et avantages touristiques) et les équipes techniques souhaitant simuler de nombreux utilisateurs dans un cadre de test de performance.

1.1 Objectifs du projet

Les objectifs du projet TourGuide sont les suivants :
- Corriger les anomalies existantes : le test TestRewardsService.nearAllAttraction() échoue par intermittence à cause d'une ConcurrentModificationException.

- Ajout d'une nouvelle fonctionnalité permettant à l'utilisateur d'obtenir les 5 attractions les plus proches de sa dernière localisation

- Amélioration les performances du système, en particulier lors de la mise à jour de la localisation des utilisateurs et le calcul des points de récompense (pour 100 000 utilisateurs : moins de 15 minutes pour l'obtention de la localisation et moins de 20 minutes pour le calcul des récompenses).

- Mise en place d'une chaîne pipeline d'intégration continue (CI) afin d'automatiser la compilation, l'exécution des tests et la génération du JAR exécutable du projet.

1.2 Hors du champ d'application

Ce projet ne couvre pas :
- L'interface front-end complète pour les utilisateurs finaux.
- La persistance dans une base de données externe (toutes les données sont en mémoire pour l'instant).
- La livraison continue.

1.3 Mesures du projet

Le succès du projet est mesuré par la fiabilité et la performance de l'application :
- Tous les tests unitaires s'exécutent avec succès, garantissant la stabilité des fonctionnalités existantes et des correctifs apportés.
- Les tests de performance confirment une amélioration du temps de traitement pour la géolocalisation et le calcul des récompenses (pour 100 000 utilisateurs : moins de 15 minutes pour l'obtention de la localisation et moins de 20 minutes pour le calcul des récompenses).

- La pipeline d'intégration continue (CI) permet de vérifier automatiquement la compilation, l'exécution des tests et le packaging du JAR sans erreur, assurant ainsi une validation complète avant chaque livraison.

2. Spécifications fonctionnelles

Gestion des utilisateurs

- Création et gestion d'un utilisateur avec un identifiant unique, nom, email et numéro de téléphone.
- Suivi de la dernière localisation de chaque utilisateur.
- Historisation des lieux visités par chaque utilisateur.
- Possibilité d'ajouter ou supprimer des utilisateurs internes en mémoire

Géolocalisation

- Récupération de la position actuelle de l'utilisateur via le service GpsUtil
- Enregistrement automatique des nouvelles positions dans l'historique de l'utilisateur
- Localisation simultanée de tous les utilisateurs simultanément grâce à un traitement asynchrone pour optimiser les performances.

Gestion des récompenses

- Calcul des récompenses associées aux attractions touristiques proches.
- Attribution de points via le service externe RewardCentral.
- Vérification automatique des distances entre les attractions et les lieux visités.
- Eviter la duplication, des récompenses déjà obtenues grâce à un contrôle des doublons.
- Traitement asynchrone et multi utilisateur du calcul des récompenses.

Recommandation d'attractions

- Calcul des 5 attractions les plus proches de la dernière position de l'utilisateur.
- Retour d'informations détaillées pour chaque attractions (nom, coordonnées distance avec l'utilisateur, nombre de point de récompense potentiels).

Gestion des offres

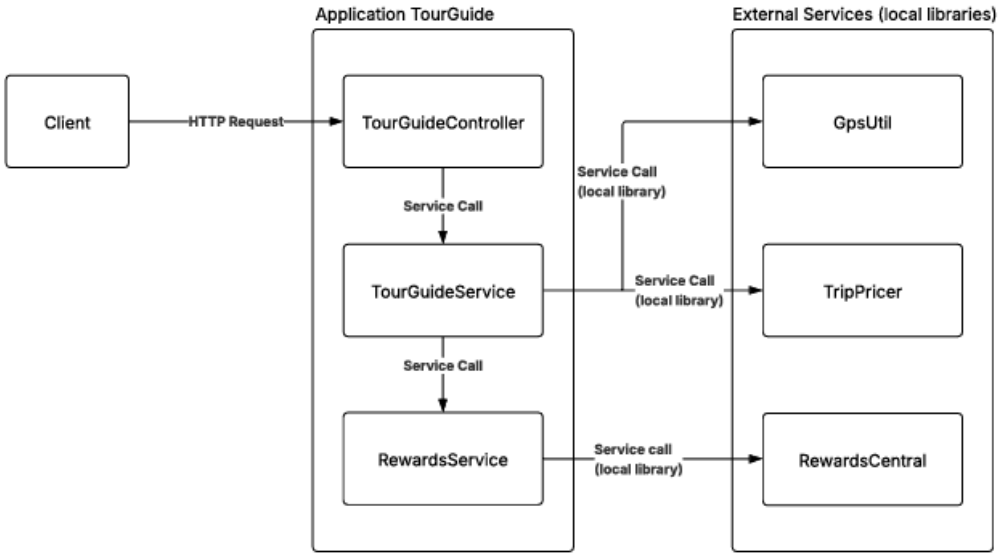
- Intégration avec le service externe TripPricer pour récupérer les offres de séjours et activités.
- Prise en compte des préférences de l'utilisateur (nombre d'adultes/enfants,

durée du voyage, ...).

- Calcul du prix en fonction des points de récompense cumulés.
- Enregistrement de la liste des offres dans le profil utilisateur.

3. Spécifications techniques

3.1 Schémas de conception technique



Ce schéma représente l'architecture technique de l'application TourGuide.
L'application suit une architecture en couches, avec :

- **TourGuideController** : couche de présentation qui expose les endpoints REST.
- **TourGuideService** : cœur de la logique applicative, qui orchestre les appels aux services.
- **RewardsService** : module dédié au calcul des récompenses liées aux attractions.
- **GpsUtil**, **TripPricer**, **RewardCentral** : services externes utilisés pour récupérer les positions GPS, les offres touristiques et les points de récompense.

Les flèches représentent les appels entre les composants.

3.2 Glossaire

- User : représente un utilisateur de l'application avec tous ses attributs
- UserPreferences: Préférences de l'utilisateur (nombre d'adultes/enfants, durée du voyage)
- UserRewards : Liste des récompenses attribuées à l'utilisateur (contenant la localisation de l'utilisateur, attraction touristique, nombre de points de récompense)
- TripDeals: Liste des offres de l'utilisateur.
- Attraction : Lieu touristique contenant un nom, une adresse, une position géographique et un identifiant unique
- VisitedLocation : Localisation visitée par un utilisateur, composée d'une position GPS et d'une date de visite
- NearbyAttractionDTO : Data Transfer Object conçu pour l'affichage des 5 attractions les plus proches, regroupant le nom, la localisation, la distance et le nombre de points de récompenses.
- TourGuideController : Couche de présentation de l'application. Elle est responsable de recevoir les requêtes HTTP, d'appeler la logique métier via TourGuideService et de renvoyer la réponse au format JSON.
- TourGuideService : Couche de service/logique métier. Elle gère les utilisateurs, orchestre les appels aux service externes (gpsUtil, tripPricer et rewardCentral) et implémente la logique de recommandation d'attractions.
- RewardsService : Service dédié au calcul des récompenses. Il utilise le service Externe RewardCentral pour obtenir des points et gère la logique de proximité entre les attractions et les lieux visités par l'utilisateur.
- GpsUtil : Bibliothèque locale (dépendance) fournissant la fonctionnalité de géolocaliser les utilisateurs et attractions.
- TripPricer : Bibliothèque locale (dépendance) utilisée pour simuler la récupération des offres de voyages et séjours pour un utilisateur.
- RewardCentral : Bibliothèque locale (dépendance) utilisée par le RewardsService pour obtenir le nombre de points de récompense associés à une attraction donnée et à un utilisateur.

3.3 Solutions techniques

<u>Amélioration de la fiabilité et des données</u>

L’objectif était de corriger la ConcurrentModificationException qui provoquait l’échec des tests par intermittence. Cette anomalie était due à la modification simultanée des listes d’attractions ou de localisation par plusieurs threads.
La solution technique retenue est l’implémentation de structures de données thread-safe :
- ` CopyOnWriteArrayList ` : Dans la méthode RewardsService.calculateRewards() pour la liste des lieux visités par un utilisateur. Cette structure crée une copie de la liste lors d’une modification, assurant que les threads en lecture accèdent à une version cohérente des données. Cette stabilisation est le prérequis indispensable pour pouvoir lancer les mécanismes de parallélisation massive sans risque de défaillance technique.
- Contrôle des doublons : Dans la méthode RewardsService.calculateRewards(), on utilise un ConcurrentHashMap.newKeySet() pour suivre récompenses déjà attribuées. Cela assure que le contrôle des doublons de UserRewards se déroule de manière thread-safe, empêchant l’attribution multiple des mêmes points de récompense lors du traitement parallèle.

<u>Recommandation des 5 attractions les plus proches</u>
L’objectif était d’implémenter une nouvelle fonctionnalité de recommandation des 5 attractions les plus proches exposées via l’endPoint TourGuideService.getNearbyAttractions().
On commence par récupérer la dernière position géographique (VisitedLocation) de l’utilisateur. Le service utilise la fonction de calcul de Distance du RewardsService. Les attractions sont ensuite triées en fonction de cette distance par rapport à l’utilisateur, les 5 premières attractions sont sélectionnées et les informations requises sont agrégées dans un NearbyAttractionDTO avant d’être envoyées au client.

<u>Optimisation de la performance</u>
L’objectif est de pouvoir obtenir la position de 100 000 utilisateurs en moins de 15 minutes, et d’obtenir les récompenses pour 100 000 utilisateurs dans un délai de 20 minutes.
Un mécanisme de traitement asynchrone et de parallélisation a été mis en place :
- Parallélisation des tâches : Utilisation d’ExecutorService avec des pools de threads dédiés dans le TourGuideService et RewardsService, ce qui permet de distribuer la charge de travail sur plusieurs cœurs de processeur.
- Localisation multi-utilisateur : la méthode trackAllUserLocation(List) utilise des objets CompletableFuture pour lancer la mise à jour de la localisation de chaque utilisateur simultanément et de manière non bloquante sur le pool de threads.
- Le calcul des récompenses multi-utilisateurs : la fonction RewardsService.calculateRewardsForAllUsers() parallélise également le calcul des récompenses à l’aide d’un pool de thread configurable, ce qui réduit le temps de

traitement global lors de la montée en charge. La méthode calculateRewardsForAllUsers utilise CompletableFuture.allOf().join() pour garantir la synchronisation du thread appelant, assurant ainsi que le test de performance mesure le temps d'exécution complet et total du lot.

Mise en place de l'intégration continue (CI)
Afin d'assurer la qualité logicielle et la stabilité des livrables, un pipeline d'intégration continue a été établie avec GitHub Actions. Cette chaîne automatise le cycle de validation suivant :
1. Compilation du code source
2. Exécution de tous les tests unitaires, assurant que les corrections d'anomalies sont validées
3. Génération d'un JAR exécutable et d'un artéfact téléchargeable à chaque livraison.

3.4 Autres solutions non retenues

Option : Le développement interne des dépendances (GpsUtil)
Raison du rejet : Cela aurait représenté une demande de développement et de budget trop élevée.

Option : Transformation des services externes en des service web permettant le load balancing.
Raison du rejet : Complexité et forte demande de développement (entre l'écriture, le déploiement et la maintenance des 3 services web).